
TRISDUCTION CORE STATED IN FORTRAN AND IN NOTHING ELSE

A Verdict Kernel That Determines Totally, Reads Its Rows, and Its Exact Scope

SHORT COMMUNICATION

A geometric verdict kernel stated at theorem grade, its one constitutive blindness scoped, and its executable proof of load

MOHAMMAD F. ISLAM, PHD¹

¹ Independent Researcher

7 September 2026

ABSTRACT. A verification kernel reads three warrant rows over N contexts, forms their correlation Gram R , and reports whether the three axes enclose volume. We state what that report determines and what it does not, at the grade each claim carries. Three independent axes intersect in exactly one point: nondegeneracy is linear independence, so the determination is total about its object and no weaker reading survives. The kernel's scalar $\det R = \lambda^2$ is nonetheless blind to the sign of λ , and we show that this blindness is a fact about direction and never about the completeness of the determination, since the intersection is equally unique under every axis reflection. What varies between cases is not the strength of the lock but the provenance of its rows, and we give the three-way partition that follows: rows that are the closure itself, an axis left unpopulated so that no point is determined, and rows furnished by the world. A single-file Fortran implementation carries the theorems as procedures and its own battery of 248 computed checks as the proof of load; every figure quoted here was produced by that program at the stated build.

KEYWORDS Gram determinant, quaternion triple product, orientation blindness, rank tolerance, verification kernel, executable proof

1. STATEMENT

Let $q_1, q_2, q_3 \in \mathbb{R}^N$ be three rows, each centred, standardised, and normalised to unit length. Write Q for the $3 \times N$ matrix with these rows and

$$R = QQ^T \in \mathbb{R}^{3 \times 3}, \quad R_{ij} = \langle q_i, q_j \rangle,$$

for their correlation Gram. Let B be an orthonormal basis of the row span, fixed once, and let $c_i \in \mathbb{R}^3$ be the coordinates of q_i in B . Read each c_i as a pure quaternion $\hat{q}_i = 0 + c_{i1}\mathbf{i} + c_{i2}\mathbf{j} + c_{i3}\mathbf{k}$ and set

$$\lambda = \text{Re}(\hat{q}_1 \hat{q}_2 \hat{q}_3).$$

Three facts are used throughout and none is new; the contribution is the arrangement and the typing. First, $\lambda = -\det[c_1; c_2; c_3]$, the scalar part of a product of three pure quaternions being the negative of the scalar triple product; reading the coordinates as quaternions is licensed because a three-axis algebra closed under an associative product with a multiplicative norm completes uniquely to $\mathbb{H}$ [1], and that norm is multiplicative [2], so $|\hat{q}_1 \hat{q}_2 \hat{q}_3| = 1$ and $|\lambda| \leq 1$. Second, $R = CC^T$ with $C = [c_1; c_2; c_3]$, so $\det R = \lambda^2$. Third, $0 \leq \det R \leq 1$, positive semi-definiteness below and Hadamard's inequality on unit rows above [3].

We call the event $\det R > 0$ a **lock**. In exact arithmetic this is the event $\text{rank } C = 3$, and Section 5 says how the implementation reads that one event at finite precision, where the two proxies can part and neither is allowed to decide alone. The paper is about what a lock determines.

2. THE DETERMINATION IS TOTAL

Read the three rows as normals of three planes through the origin of $\mathbb{R}^3$. Their intersection is the null space of C , of dimension $3 - \text{rank } C$.

Theorem 1 (uniqueness). If $\det R > 0$ then $\text{rank } C = 3$, the intersection has dimension zero, and the three planes meet in exactly one point. If $\text{rank } C = 2$ the intersection is a line, if $\text{rank } C = 1$ a plane, and if $\text{rank } C = 0$ all of $\mathbb{R}^3$; in none of these is a point determined.

Proof. $\det R = (\det C)^2 > 0$ if and only if C is invertible, if and only if its null space is trivial. The null space has dimension $3 - \text{rank } C$, a line, a plane, or the whole space as the rank falls. $\square$

The theorem is linear algebra and carries theorem grade. What it forbids is worth stating positively, because the vocabulary of verification tends to soften it. There is no second point. There is no interval of

admissible points. Nothing about the determination is provisional. A lock is always a determination and is total about its object, and its object is its intersection.

The word *permission* has been used for the lock elsewhere in this programme, on the ground that a lock does not settle the proposition whose rows were read. That ground is correct and the word is wrong. It grades the lock, and locks carry no strengths. What differs between cases is what the rows were, which is Section 4.

3. THE ONE BLINDNESS, AND ITS SCOPE

Let $D = \text{diag}(\pm 1, \pm 1, \pm 1)$ implement any pattern of axis reflections.

Theorem 2 (orientation blindness). $\det(DRD) = \det R$ for every D , while λ changes sign under every odd pattern.

Proof. $\det(DRD) = \det(D)^2 \det R = \det R$. Reflecting one row negates one c_i and hence the triple product. $\square$

The scalar the kernel reports cannot tell a proposition from its negation. This is real and it is the only blindness the kernel has. Its scope is exact: the blindness enters at the single operation $\lambda \mapsto \lambda^2$ and nowhere else. It is a fact about *direction*. It is not a fact about *completeness*: by Theorem 1 the intersection is equally unique under every reflection, so the determination is untouched by the operation that discards the sign. Direction is recovered from the rows themselves, never from the scalar, and in the implementation it is recovered from a linguistic seal that precedes the kernel and from the signed λ carried out of band.

At the executed build the reflection is exact and not merely small. Under strict IEEE arithmetic [5] with reassociation disabled, negating one row negates every product and every sum bit for bit, and the power-of-two scaling used in the quaternion product commits no rounding, so the assertion $\lambda(\neg P) = -\lambda(P)$ is tested with exact equality and holds on three independent builds. A tolerance there would concede rounding the arithmetic does not commit.

4. WHAT THE ROWS WERE

Since the lock is total about its intersection, the only remaining question is what that intersection is an intersection of. Three cases exhaust it and none is a weaker lock.

Closure-rowed. The rows are the closure of the proposition's own terms: definitions fixed by stipulation and finite deductive objects delivered in hand. Then determining the rows and determining the proposition are one determination, not two, because there is no second thing for a gap to open between. The lock reaches the proposition. This is the case of a theorem of the algebra above: $\text{Re}(\mathbf{ijk}) = -1$ is closure-rowed.

Unpopulated. One axis carries no independent con-

tent. Then $\text{rank } C \leq 2$, the intersection is a line at best, and no lock forms. This is not a weak lock; it is the absence of one. An open conjecture over an unbounded domain with no proof in hand sits here.

World-rowed. The rows were furnished by measurement. The lock determines its intersection uniquely given those rows, and every revisability lives in the rows and none of it in the lock. A measurement that is later revised furnishes different rows; the lock on the old rows was never wrong about them.

The partition is structural rather than theorem-grade: it names the provenance of rows and not a property of the algebra. Its one theorem-grade leg is Theorem 1, which is what licenses saying that the second case determines nothing at all.

5. READING RANK AT FINITE PRECISION

The implementation decides existence by dimension and not by determinant, because the dimension is the quantity Theorem 1 speaks of and the determinant is a magnitude read off it. Rank is read from the eigenvalues $w_1 \geq w_2 \geq w_3$ of R , computed by cyclic Jacobi sweeps [4] on a copy scaled by its largest entry, against

$$\tau = 8n\epsilon \max_i |w_i|,$$

with $n = 3$ and ϵ the unit roundoff; the factor $8n\epsilon$ is the standard backward-error bound on a symmetric eigenvalue [6], beneath which an eigenvalue is not separable from rounding at this precision. The tolerance is relative to the spectrum and carries no absolute floor. An earlier draft pinned a floor of one, and under it $R = 10^{-20}I$ returned rank zero where it is three; rank had become a fact about units. The relative form is invariant under uniform positive scaling of R , which is what a rank should be.

Two floors are read on R : the collapse floor $\epsilon_c = 100\epsilon N$ on $\det R$ and the conditioning gate on $\kappa(R) = w_1/w_3$. Their separation is known in closed form. Among correlation matrices of trace three at conditioning κ , the determinant is minimised by the positive-euicorrelation matrix and equals

$$\det R_{\min}(\kappa) = \frac{27\kappa}{(\kappa + 2)^3},$$

so the two floors cannot contend while $\kappa < \sqrt{27/(100\epsilon N)}$, which at $N = 24$ is 7.1×10^6 , above the gate at 10^6 . Within that envelope the verdict boundary is decided by structure and not by rounding. Where the rank floor and the collapse floor nonetheless disagree, the implementation neither declares the lock broken nor declares it determined; it reports the arithmetic as incomplete and re-runs at higher precision. A floor disagreement escalates and never decides.

6. FALSIFICATION

Three criteria, each aimed at one theorem of this paper and each cheap to run against the shipped program.

1. Exhibit three independent unit rows whose planes meet in more than one point, or in an interval. This falsifies Theorem 1 and with it the spine.
2. Under a strict IEEE build with reassociation disabled, exhibit one row reflection under which the composed scalar fails $\lambda(\neg P) = -\lambda(P)$ bit for bit. This falsifies the exactness claim of Section 3.
3. Exhibit a positive semi-definite Gram whose computed rank changes under uniform positive scaling within the representable range. This falsifies the scale invariance of Section 5.

7. EXECUTED RECEIPTS

Every figure below was printed by the shipped program, built as `-std=f2018 -O2 -fno-fast-math -ffp-contract=off`, and is local execution evidence gated against its bound; the identity residual varies by a few ulp between builds, three ulp across the three builds used here, and every value passes the printed bound.

On a three-axis warrant matrix over $N = 24$ contexts, intersection dimension 0, $\det R = 0.844905725419$, $|\lambda^2 - \det R| = 1.110 \times 10^{-16}$ against the conditioning-scaled bound 1.996×10^{-15} , rank margin 8.350×10^{13} . With the third axis made a combination of the first two, intersection dimension 1, verdict broken, and the message names the geometry: rank two, the intersection is a line and not a point.

Under full negation, $\lambda(P) = -0.919187535500$ and $\lambda(\neg P) = +0.919187535500$, with $\max |G(P) - G(\neg P)| = 0.000 \times 10^0$ and $|\det R(P) - \det R(\neg P)| = 0.000 \times 10^0$ exactly. The Hamilton relation returns $\text{Re}(\mathbf{ijk}) = -1.000000000000$. On the orthonormal Fourier triad $\{\sin t, \cos t, \sin 2t\}$ the lock reaches its ceiling, $\det R = 1.000000000000$ and $|\lambda| = 1.000000000000$.

The battery of 248 computed checks, eight classes over every public procedure including hostile non-finite and extreme-magnitude input, returned zero failures on three builds: strict IEEE with floating-point traps armed and signalling-NaN initialisation, optimised, and optimised with contraction disabled. The program stops with a nonzero exit if any check fails, so a binary that prints its final line has passed.

8. CONCLUSION

A lock determines its intersection completely and uniquely, and the one thing it cannot read is the sign. Those are two different facts, and conflating them produced a vocabulary that graded locks by strength. Locks carry no strengths. The compartment names what the rows were, and only in the closure-rowed

case does the determination reach the proposition, because only there are the rows and the proposition the same thing.

9. PROVENANCE AND METHOD DISCLOSURE

This paper was developed under Trisduction, a verification and organizing discipline that adds the cited results no warrant; the method and its executable batteries are stated in full at the reference below.

10. REFERENCES

- [1] Frobenius, F. G. (1878). Über lineare Substitutionen und bilineare Formen. *J. reine angew. Math.* 84, 1–63.
- [2] Hurwitz, A. (1898). Über die Composition der quadratischen Formen von beliebig vielen Variablen. *Nachr. Ges. Wiss. Göttingen*, 309–316.
- [3] Hadamard, J. (1893). Résolution d’une question relative aux déterminants. *Bull. Sci. Math.* 17, 240–246.
- [4] Jacobi, C. G. J. (1846). Über ein leichtes Verfahren die in der Theorie der Säcularstörungen vorkommenden Gleichungen numerisch aufzulösen. *J. reine angew. Math.* 30, 51–94.
- [5] IEEE Std 754-2019. *IEEE Standard for Floating-Point Arithmetic*. IEEE, 2019.
- [6] Higham, N. J. (2002). *Accuracy and Stability of Numerical Algorithms*, 2nd ed. SIAM.
- [7] Islam, M. F. (2026). TRISDUCTION Core Thesis in Fortran, v2.4.0. protocols/Executable Thesis/ in the repository 1000sapients/Trisduction, branch main. SHA-256 head b5d12f7326e196c2.
- [8] Islam, M. TRISDUCTION: A Linguistically, Topologically, and Mathematically Sealed Verification Architecture. Triaxial Orthogonality, Twelve-Gate Closure, the Quaternionic Completion, the Root Axiom, and the Master Pre-Sealed Proposition Ledger. Zenodo, version 4, 19 June 2026. DOI 10.5281/zenodo.20757507. <https://zenodo.org/records/20757507>. Mirror: PhilArchive record ISLTTG, <https://philpapers.org/rec/ISLTTG>. Master reference, continuously updated at the same location.

APPENDIX A. THE EXECUTABLE, IN FULL

The single-file Fortran thesis that produced every figure in this paper, listed verbatim. 1347 lines. SHA-256 b5d12f7326e196c2a017fe57f1eae45de66708528d1252913cd4dce6991017. It is byte-identical to protocols/Executable Thesis/TRISDUCTION_Core_Thesis_Fortran_v2_4_0.f90 on branch main of 1000sapient/Trisduction. Build and run:

```
gfortran -std=f2018 -O2 -fno-fast-math -ffp-contract=off TRISDUCTION_Core_Thesis_Fortran_v2_4_0.f90 -o
thesis && ./thesis
```

A binary that prints its final line has passed its own 248-check battery.

```
!=====
! T R I S D U C T I O N .   R A M
! THE CORE THESIS, STATED IN FORTRAN AND IN NOTHING ELSE
!
! The thesis is not described here. It is executed. Every claim that can be
! made to run is compiled as a procedure, and the claims that cannot run are
! compiled as the gates that refuse to emit without them.
!
! Three independent axes intersect in exactly one point. Nondegeneracy is
! linear independence. There is no second point and no interval. A LOCK IS
! ALWAYS A DETERMINATION AND IS TOTAL ABOUT ITS OBJECT, and its object is its
! intersection. Nothing below grades a lock, because locks carry no strengths.
! What differs between cases is what the ROWS were, and the compartment names
! that and nothing else.
!
! I   CLOSURE-ROWED   rows are the closure; the two determinations are one
! II  UNPOPULATED    an axis is empty; two planes meet in a line; no lock forms
! III WORLD-ROWED    rows furnished by the world; revisability lives in the rows
!
! Two registers, sequential load. Register A, the kinetic preloader on RA,
! boots first: Seal L the Tongue, then Seal G the Form and Seal M the Number.
! Register B, the computational kernel on RAM, loads second on that discipline.
! Bedrock precedence: L then G then M. The Number reads magnitude on rows it
! did not author. Every compartment is RA-conditional at the act.
!
! The file carries its own battery and runs it last: eight fuzz classes,
! seven over the numerical kernel and one over every symbolic and router
! procedure, computed checks throughout, an oracle that stops the program
! on any failure. A binary that reaches its final line has
! passed. Nothing below is narrated; every figure is computed in this run.
!
! Delta-M = 0. No new mathematics is authored by this file.
!=====

!=====

module trisduction_rows
  use, intrinsic :: ieee_arithmetic, only: ieee_is_finite, ieee_value, ieee_quiet_nan, ieee_positive_inf
  implicit none
  private
  integer, parameter, public :: dp = selected_real_kind(15, 307)

  !--- THE VERDICT ECONOMY. Three states, native. There is no fourth. -----
  integer, parameter, public :: SEALED = 1, BROKEN = 2, OPENV = 3
  integer, parameter, public :: XIO = 4, OHO = 5           ! refinements INSIDE openness
  integer, parameter, public :: DETERMINED = 6           ! the kernel's own token
  integer, parameter, public :: DOTMARK = 7              ! outside the economy

  !--- THE THREE COMPARTMENTS. Not strengths. Kinds of row. -----
  integer, parameter, public :: CLOSURE_ROWED = 1, UNPOPULATED = 2, WORLD_ROWED = 3
  integer, parameter, public :: NOT_COMPARTMENTED = 0

  public :: snorm, finite_vec, finite_mat, lambda_of_admissible
  public :: det3_representable
  public :: token, token_class, compartment_name, qmul, det3, jacobi3, cond_sym3
  public :: prepare_rows, span_basis, lambda_of, kernel_determine
  public :: intersection_dim, seal_L, drill_screen, row_cascade
  public :: ground_dim, route_halt, compartment_grade
contains

  ! THE FINITE-INPUT CONTRACT. Every public numeric routine below reads only
  ! finite input. A NaN or an infinity is not a quantity, and a routine that
  ! returns a number or a true predicate on one has manufactured a quantity
  ! from a non-quantity. Pure numeric functions return NaN or .false. on
  ! violation, because a NaN in yielding a NaN out is honest and a zero would
  ! not be. Predicate and preparation subroutines report refusal through
  ! ok, verdict, or reason, because there a non-quantity is a data condition.
  ! Low-level computational subroutines whose preconditions are violated
  ! halt, because there the violation is a programming fault.
  pure function finite_vec(x) result(ok)
    real(dp), intent(in) :: x(:)
    logical :: ok
    ok = all(ieee_is_finite(x))
  end function finite_vec
```

```

pure function finite_mat(A) result(ok)
  real(dp), intent(in) :: A(:, :)
  logical :: ok
  ok = all(ieee_is_finite(A))
end function finite_mat

! The scaled two-norm. Squaring before scaling overflows for any row near the
! representable ceiling, so the largest magnitude comes out first and the
! square is formed on values bounded by one. Exact for a zero row.
pure function snorm(x) result(r)
  real(dp), intent(in) :: x(:)
  real(dp) :: r, m
  if (.not. finite_vec(x)) then
    r = ieee_value(1.0_dp, ieee_quiet_nan); return
  end if
  m = maxval(abs(x))
  if (m > 0.0_dp) then
    r = m*sqrt(sum((x/m)**2))
  else
    r = 0.0_dp
  end if
end function snorm

pure function token(v) result(s)
  integer, intent(in) :: v
  character(len=30) :: s
  select case (v)
    case (SEALED);      s = '[seal] SEALED'
    case (BROKEN);      s = '[X]   BROKEN'
    case (OPENV);       s = '[?]   UNDER-DETERMINED'
    case (XIO);         s = '[Xio]  TERMINAL SUSPENSION'
    case (OHO);         s = '[OO]   GROUNDED-SEALED HALT'
    case (DETERMINED);  s = '[LOCK] DETERMINED'
    case (DOTMARK);     s = '[.]    chosen silence'
    case default;       s = '[??]   untyped'
  end select
end function token

! The economy is three-state. XIO and OHO are refinements INSIDE the
! openness and never a fourth state; DETERMINED is the kernel's geometric
! token read before any register interprets it; the dot stands outside the
! economy entirely and is never census-counted. A caller that needs to know
! whether a token is a verdict state asks, and is not left to assume.
pure function token_class(v) result(s)
  integer, intent(in) :: v
  character(len=26) :: s
  select case (v)
    case (SEALED, BROKEN, OPENV); s = 'verdict state'
    case (XIO, OHO);             s = 'refinement inside openness'
    case (DETERMINED);           s = 'kernel geometric token'
    case (DOTMARK);              s = 'outside the economy'
    case default;                s = 'untyped'
  end select
end function token_class

pure function compartment_name(c) result(s)
  integer, intent(in) :: c
  character(len=34) :: s
  select case (c)
    case (CLOSURE_ROWED); s = 'I   CLOSURE-ROWED  [seal A . RA]'
    case (UNPOPULATED);   s = 'II  UNPOPULATED   no lock forms'
    case (WORLD_ROWED);   s = 'III WORLD-ROWED  rows revisable'
    case default;         s = '--  not compartmented'
  end select
end function compartment_name

!=====
! THE ALGEBRA. Seal M loads LAST and reads magnitude on rows it did not
! author. It is constitutively unable to read sign, and that is a fact
! about DIRECTION, never about the completeness of the determination.
!=====

! Hamilton product. The substrate chirality lives here: i*j*k = -1.
! Both factors are scaled by their own largest magnitude, the product is
! formed on components bounded by one, and the scale is restored only after
! the restored magnitude has been checked in log space. Representability
! is therefore decided on the actual components and not on the norm bound
! |ab| = |a||b|, which is sufficient for refusal but not necessary, since a
! component can be smaller than the norm. Unrepresentable is NaN, never a
! saturated or wrapped quantity.
pure function qmul(a, b) result(c)
  real(dp), intent(in) :: a(4), b(4)
  real(dp) :: c(4), aa(4), bb(4), cc(4), sa, sb, lg
  if (.not. (finite_vec(a) .and. finite_vec(b))) then
    c = ieee_value(1.0_dp, ieee_quiet_nan); return
  end if
  sa = maxval(abs(a)); sb = maxval(abs(b))
  if (.not. (sa > 0.0_dp .and. sb > 0.0_dp)) then

```

```

    c = 0.0_dp; return
end if
aa = a/sa; bb = b/sb
cc(1) = aa(1)*bb(1) - aa(2)*bb(2) - aa(3)*bb(3) - aa(4)*bb(4)
cc(2) = aa(1)*bb(2) + aa(2)*bb(1) + aa(3)*bb(4) - aa(4)*bb(3)
cc(3) = aa(1)*bb(3) - aa(2)*bb(4) + aa(3)*bb(1) + aa(4)*bb(2)
cc(4) = aa(1)*bb(4) + aa(2)*bb(3) - aa(3)*bb(2) + aa(4)*bb(1)
if (.not. (maxval(abs(cc)) > 0.0_dp)) then
    c = 0.0_dp; return
end if
lg = log(sa) + log(sb) + log(maxval(abs(cc)))
if (lg >= log(huge(1.0_dp))) then
    c = ieee_value(1.0_dp, ieee_quiet_nan); return
end if
! The scale is restored in binary, not through exp(log). The mantissas
! fraction(sa) and fraction(sb) lie in [1/2, 1), so their product times cc
! is bounded by four and cannot overflow, and scale() by the summed
! exponent is an exact power-of-two multiply. exp(log(sa)+log(sb)) loses
! about thirteen digits here, since an ulp of a log near 709 is 1e-13
! absolute and becomes 1e-13 relative on the result; the battery caught
! that and this is the repair.
c = scale(fraction(sa)*fraction(sb)*cc, exponent(sa) + exponent(sb))
end function qmul

! The triple products overflow for entries near the ceiling even where the
! determinant is finite after cancellation, so the matrix is scaled first and
! the cube of the scale is restored once.
! Purity is kept, so the answer to "is this value a quantity" is a separate
! predicate rather than an out argument. Where the computed determinant
! magnitude exceeds the representable normal range det3 saturates and det3_representable returns
! .false.; a caller that reads the saturated value as a quantity has been
! told otherwise and did not ask.
! Equilibration, not a single global scale. One maximum over the whole
! matrix lets a small diagonal entry underflow to zero in the normalised
! copy, so diag(1e200, 1e200, 1e-200) returned zero where the determinant
! is 1e200 and diag(1e308, 1e308, 1e-20) was certified representable where
! it is not. Rows are scaled by their own maxima, then columns by theirs,
! and the determinant of the equilibrated core is reassembled in log space
! so representability is decided before any product is formed.
pure subroutine det3_core(M, core, lg, sgn, ok)
    real(dp), intent(in) :: M(3,3)
    real(dp), intent(out) :: core, lg, sgn
    logical, intent(out) :: ok
    real(dp) :: N(3,3), r(3), c(3)
    integer :: i
    ok = finite_mat(M); core = 0.0_dp; lg = -huge(1.0_dp); sgn = 0.0_dp
    if (.not. ok) return
    N = M
    do i = 1, 3
        r(i) = maxval(abs(N(i,:)))
        if (r(i) > 0.0_dp) N(i,:) = N(i,+)/r(i)
    end do
    do i = 1, 3
        c(i) = maxval(abs(N(:,i)))
        if (c(i) > 0.0_dp) N(:,i) = N(:,i)/c(i)
    end do
    core = N(1,1)*N(2,2)*N(3,3) - N(2,3)*N(3,2) &
        - N(1,2)*N(2,1)*N(3,3) - N(2,3)*N(3,1) &
        + N(1,3)*N(2,1)*N(3,2) - N(2,2)*N(3,1)
    if (.not. (abs(core) > 0.0_dp)) return ! computed zero; lg stays -huge
    sgn = sign(1.0_dp, core)
    lg = log(abs(core))
    do i = 1, 3
        if (r(i) > 0.0_dp) lg = lg + log(r(i))
        if (c(i) > 0.0_dp) lg = lg + log(c(i))
    end do
end subroutine det3_core

pure function det3(M) result(d)
    real(dp), intent(in) :: M(3,3)
    real(dp) :: d, core, lg, sgn
    logical :: ok
    call det3_core(M, core, lg, sgn, ok)
    if (.not. ok) then
        d = ieee_value(1.0_dp, ieee_quiet_nan); return
    end if
    if (.not. (abs(core) > 0.0_dp)) then
        d = 0.0_dp; return
    end if
    ! At the ceiling itself exp(lg) can round past huge and manufacture an
    ! overflow where a saturated finite value was promised, so the boundary
    ! saturates too.
    if (lg >= log(huge(1.0_dp))) then
        d = sign(huge(1.0_dp), sgn); return ! saturating; ask det3_representable
    end if
    if (lg < log(tiny(1.0_dp))) then
        d = sign(0.0_dp, sgn); return ! below the normal range; ask
    end if
    d = sgn*exp(lg)

```

```

end function det3

! True when det3 returns a non-saturated, non-underflowed normal quantity
! under this floating computation, or when the equilibrated core computes
! as zero. A computed zero is not certified exact singularity.
pure function det3_representable(M) result(r)
  real(dp), intent(in) :: M(3,3)
  logical :: r
  real(dp) :: core, lg, sgn
  logical :: ok
  call det3_core(M, core, lg, sgn, ok)
  if (.not. ok) then
    r = .false.; return
  end if
  if (.not. (abs(core) > 0.0_dp)) then
    ! A COMPUTED zero is treated as a representable zero. This routine
    ! reads floating arithmetic and cannot tell exact singularity from a
    ! determinant that cancelled or underflowed to zero on the way; that
    ! distinction needs interval or compensated arithmetic and is not
    ! claimed here.
    r = .true.; return
  end if
  r = (lg < log(huge(1.0_dp))) .and. (lg >= log(tiny(1.0_dp)))
end function det3_representable

! CONTRACT: A is symmetric. The routine reads only what a symmetric matrix
! carries, so an asymmetric input would be silently symmetrised by the
! sweep and the eigenvalues returned would belong to no matrix the caller
! supplied. The precondition is therefore enforced, not merely documented.
subroutine jacobi3(A, w)
  real(dp), intent(in) :: A(3,3)
  real(dp), intent(out) :: w(3)
  real(dp) :: M(3,3), Asc(3,3), theta, t, c, s, tau, g, h, asym, amax
  integer :: sweep, p, q, k
  if (.not. finite_mat(A)) error stop 'jacobi3: input is not finite'
  ! The check is performed on a scaled copy, because differencing two
  ! entries near the representable ceiling overflows the test itself and a
  ! precondition guard that faults while guarding is worse than none.
  amax = maxval(abs(A))
  if (amax > 0.0_dp) then
    Asc = A/amax
  else
    Asc = A
  end if
  asym = max(abs(Asc(1,2)-Asc(2,1)), abs(Asc(1,3)-Asc(3,1)), abs(Asc(2,3)-Asc(3,2)))
  if (asym > 100.0_dp*epsilon(1.0_dp)) then
    error stop 'jacobi3: input matrix is not symmetric within tolerance'
  end if
  ! The whole sweep runs on the scaled copy and the eigenvalues are unscaled
  ! once at the end. This is what makes every interior sum, difference and
  ! square safe at any admissible magnitude, rather than guarding each one.
  M = Asc
  do sweep = 1, 60
    g = abs(M(1,2)) + abs(M(1,3)) + abs(M(2,3))
    if (g < 1.0e-300_dp) exit
    do p = 1, 2
      do q = p+1, 3
        ! The negligibility test comes FIRST, because theta overflows in
        ! its own division where the off-diagonal is tiny against the
        ! diagonal spread. An off-diagonal below the rounding of the two
        ! diagonals it couples carries no rotation, so it is zeroed and
        ! skipped rather than divided by.
        if (abs(M(p,q)) <= epsilon(1.0_dp)*max(abs(M(p,p)), abs(M(q,q)))) then
          M(p,q) = 0.0_dp; M(q,p) = 0.0_dp
        else if (abs(M(p,q)) > tiny(1.0_dp)) then
          theta = (M(q,q) - M(p,p)) / (2.0_dp*M(p,q))
          ! theta*theta overflows where the off-diagonal is negligible
          ! against the diagonal spread. The asymptotic branch is the
          ! same rotation read where the square cannot be formed.
          if (abs(theta) > sqrt(huge(1.0_dp))) then
            t = 0.5_dp / theta
          else
            t = sign(1.0_dp, theta) / (abs(theta) + sqrt(theta*theta + 1.0_dp))
          end if
          c = 1.0_dp / sqrt(t*t + 1.0_dp); s = t*c; tau = s/(1.0_dp + c)
          h = t*M(p,q)
          M(p,p) = M(p,p) - h; M(q,q) = M(q,q) + h
          M(p,q) = 0.0_dp; M(q,p) = 0.0_dp
          do k = 1, 3
            if (k /= p .and. k /= q) then
              g = M(k,p); h = M(k,q)
              M(k,p) = g - s*(h + g*tau); M(p,k) = M(k,p)
              M(k,q) = h + s*(g - h*tau); M(q,k) = M(k,q)
            end if
          end do
        end do
      end do
    end do
  end do
end subroutine jacobi3

```

```

end do
! Unscaling can exceed the representable range for a finite input whose
! eigenvalue does. The infinity is then produced by ieee_value, which
! raises no exception, and never by the multiplication, which would trap
! before any consumer could refuse the result.
! The guard is nested and not conjoined: Fortran does not promise
! short-circuit evaluation, so huge/amax must never be formed unless
! amax exceeds one, or the guard itself overflows on a tiny matrix.
do k = 1, 3
  if (amax > 1.0_dp) then
    if (abs(M(k,k)) > huge(1.0_dp)/amax) then
      w(k) = sign(ieee_value(1.0_dp, ieee_positive_inf), M(k,k))
    else
      w(k) = amax*M(k,k)
    end if
  else
    w(k) = amax*M(k,k)          ! amax <= 1: the product only shrinks
  end if
end do
end subroutine jacobi3

! CONTRACT: A is symmetric positive semi-definite, a Gram. On a general
! symmetric matrix this returns a MAGNITUDE condition number and not the
! spectral one, because the extremes are taken over absolute eigenvalues.
function cond_sym3(A) result(kap)
  real(dp), intent(in) :: A(3,3)
  real(dp) :: kap, w(3), lo, hi
  call jacobi3(A, w)
  ! A finite matrix can carry an eigenvalue beyond the representable range,
  ! since three entries near the ceiling sum to about three times it. The
  ! sweep reports that honestly as an infinity and the consumer refuses it.
  if (.not. finite_vec(w)) error stop 'cond_sym3: eigenvalues are not representable'
  ! The contract names a Gram, so definiteness is enforced rather than
  ! assumed. An indefinite matrix would return a magnitude condition number
  ! that reads like a spectral one and would be believed.
  hi = maxval(abs(w))
  if (.not. (hi > 0.0_dp)) then
    kap = huge(1.0_dp); return          ! the zero matrix has no conditioning
  end if
  ! Relative to the spectrum and carrying no absolute floor, for the same
  ! reason the rank tolerance carries none: a floor pinned at one would make
  ! definiteness a fact about units.
  if (minval(w) < -100.0_dp*epsilon(1.0_dp)*hi) then
    error stop 'cond_sym3: input is not positive semi-definite within tolerance'
  end if
  lo = minval(abs(w))
  ! The floor is tiny() and not zero: a positive denormal divisor overflows
  ! the quotient, so the guard is on representability and not on sign.
  if (lo <= tiny(1.0_dp)) then
    kap = huge(1.0_dp)
  else
    kap = hi/lo
  end if
end function cond_sym3

! THE SPINE, COMPUTED. Read the three rows as normals of planes through the
! origin. The intersection of the planes has dimension 3 - rank. Rank three
! returns dimension ZERO, one point, unique, no interval. Rank two returns
! dimension ONE, a line, and no point is determined at all.
! CONTRACT: G is a SYMMETRIC GRAM, G = Q Q^T, formed by the caller. It is not
! a matrix of raw rows. The dimension returned is 3 - rank(G), which is the
! dimension of the intersection of the three planes whose normals the rows of
! Q are; the routine reads the Gram because rank is what it needs and the
! Gram is where rank is cheap. Passing raw rows violates the contract: in
! exact arithmetic rank(Q Q^T) equals rank(Q), so the numerical dimension
! may coincide, but the routine is specified to read a symmetric PSD Gram
! and enforces that specification rather than trusting the coincidence.
! The rank tolerance is not chosen. It is the largest eigenvalue times the
! machine epsilon times the matrix order, the standard perturbation bound on
! a symmetric eigenvalue: below it an eigenvalue is not separable from
! rounding at this precision.
subroutine intersection_dim(G, d, sep)
  real(dp), intent(in) :: G(3,3)
  integer, intent(out) :: d
  real(dp), intent(out) :: sep          ! margin: smallest eigenvalue over tol
  integer :: i, rank
  real(dp) :: w(3), tol, lo, hi
  call jacobi3(G, w)
  if (.not. finite_vec(w)) error stop 'intersection_dim: eigenvalues are not representable'
  ! The tolerance is RELATIVE to the spectrum and carries no absolute floor,
  ! so rank is invariant under uniform scaling of the Gram: a Gram at 1e-20
  ! has the same rank as the same Gram at 1. An absolute floor would make
  ! rank a fact about units, which is the chart-manufactured magnitude this
  ! architecture bars from entering a verdict as structure.
  hi = maxval(abs(w))
  if (.not. (hi > 0.0_dp)) then
    d = 3; sep = 0.0_dp; return          ! the zero Gram carries no direction
  end if
  ! The contract names a Gram, so definiteness is enforced and the count is

```

```

! over POSITIVE eigenvalues. Counting magnitudes would hand an indefinite
! matrix a geometric dimension it does not have.
if (minval(w) < -100.0_dp*epsilon(1.0_dp)*hi) then
    error stop 'intersection_dim: G is not positive semi-definite within tolerance'
end if
tol = 8.0_dp * real(size(w),dp) * epsilon(1.0_dp) * hi
rank = 0; lo = huge(1.0_dp)
do i = 1, 3
    if (w(i) > tol) then
        rank = rank + 1
        lo = min(lo, w(i))
    end if
end do
d = 3 - rank
if (rank > 0) then
    sep = lo/tol
else
    sep = 0.0_dp
end if
end subroutine intersection_dim

subroutine prepare_rows(M, Q, ok)
    real(dp), intent(in) :: M(:, :)
    real(dp), intent(out) :: Q(size(M,1), size(M,2))
    logical, intent(out) :: ok
    integer :: i, n
    real(dp) :: mu, sd, nrm, scale_
    n = size(M,2); ok = .true.; Q = M
    if (.not. finite_mat(M)) then
        ok = .false.; return ! a non-quantity is not a row
    end if
    if (n <= 1) then
        ok = .false.; return ! no sample variance is defined at n = 1
    end if
    do i = 1, size(M,1)
        scale_ = maxval(abs(M(i,:)))
        ! Scale out before summing. Squaring a row near the representable
        ! ceiling overflows the sum of squares long before the variance is
        ! meaningful, so the row is divided by its own largest magnitude first
        ! and the statistic is formed on the scaled copy. The verdict is
        ! unchanged because every downstream step is scale-invariant.
        if (scale_ > 0.0_dp) Q(i,:) = Q(i,+)/scale_
        mu = sum(Q(i,:))/real(n,dp); Q(i,:) = Q(i,:) - mu
        sd = sqrt(sum(Q(i,)**2)/real(n-1,dp))
        ! The floor is relative to the row's own scale. A spread that is only
        ! rounding on a large row is not content, and normalising by it would
        ! manufacture an axis out of noise.
        if (.not. (sd > 0.0_dp) .or. sd <= real(n,dp)*epsilon(1.0_dp)) then
            ok = .false.; return
        end if
        Q(i,:) = Q(i,+)/sd
        ! A z-scored row has sum of squares exactly n-1 by construction, so
        ! this norm is sqrt(n-1) and cannot overflow for any representable n.
        ! snorm is used anyway: one norm primitive across the module is one
        ! place to get it right, and a raw sum of squares reads as a hazard
        ! to a hostile reviewer even where the bound forecloses it.
        nrm = snorm(Q(i,:))
        if (.not. (nrm > 0.0_dp)) then
            ok = .false.; return
        end if
        Q(i,:) = Q(i,+)/nrm
    end do
end subroutine prepare_rows

! The guard is mandatory and never optional, because a guard a caller may
! skip is not a guard. A residual below the rounding floor is not a direction
! and is refused rather than normalised.
subroutine span_basis(Q, B, ok)
    real(dp), intent(in) :: Q(:, :)
    real(dp), intent(out) :: B(size(Q,1), size(Q,2))
    logical, intent(out) :: ok
    integer :: i, j
    real(dp) :: v(size(Q,2)), nrm, floor_, scale_
    if (.not. finite_mat(Q)) error stop 'span_basis: input is not finite'
    ok = .true.; B = 0.0_dp
    floor_ = real(size(Q,2),dp) * epsilon(1.0_dp)
    do i = 1, size(Q,1)
        ! Each row is scaled by its own largest magnitude BEFORE any projection.
        ! Scaling a row leaves its span untouched and bounds every entry by one,
        ! so the dot products inside the loop cannot overflow. The floor is
        ! then relative to the scaled row by construction.
        scale_ = maxval(abs(Q(i,:)))
        if (.not. (scale_ > 0.0_dp)) then
            ok = .false.; return
        end if
        v = Q(i,+)/scale_
        do j = i, size(Q,2)
            v = v - dot_product(v, B(j,:))*B(j,:)
        end do
    end do

```

```

    nrm = snorm(v)
    if (.not. (nrm > floor_)) then
        ok = .false.; return
    end if
    B(i,:) = v/nrm
end do
end subroutine span_basis

! Only the first three rows of Q and B are screened, because only the first
! three rows are read; the shape test requires at least three rows
! beyond them are not consulted by lambda_of at all.
! The admission test is a pure predicate so it can be exercised in-process
! on hostile input without halting. It is the single source of truth for
! what lambda_of accepts; lambda_of calls it and halts on refusal. The
! order inside is load-bearing: unit-ness is proven by the scaled norm
! BEFORE any dot product forms, so every dot product below runs on rows
! whose components are O(1) under the admission tolerance and never
! O(huge), and the dot products cannot overflow.
pure subroutine lambda_of_admissible(Q, B, ok, reason)
    real(dp), intent(in) :: Q(:, :), B(:, :)
    logical, intent(out) :: ok
    character(len=*) , intent(out) :: reason
    real(dp) :: btol
    integer :: i, j
    ok = .false.; reason = ''
    if (size(Q,1) < 3 .or. size(B,1) < 3 .or. size(Q,2) /= size(B,2) .or. size(Q,2) < 3) then
        reason = 'needs Q and B with at least three rows, at least three columns, and equal column counts'
        return
    end if
    if (.not. (finite_mat(Q(1:3,:)) .and. finite_mat(B(1:3,:)))) then
        reason = 'input is not finite'; return ! on the rows read, and no others
    end if
    btol = 100.0_dp * real(size(B,2),dp) * epsilon(1.0_dp)
    do i = 1, 3
        if (abs(snorm(B(i,:)) - 1.0_dp) > btol) then
            reason = 'B rows are not unit'; return
        end if
        if (abs(snorm(Q(i,:)) - 1.0_dp) > btol) then
            reason = 'Q rows are not unit'; return
        end if
    end do
    do i = 1, 3
        do j = i+1, 3
            if (abs(dot_product(B(i,:), B(j,:))) > btol) then
                reason = 'B rows are not orthogonal'; return
            end if
        end do
    end do
    ok = .true.
end subroutine lambda_of_admissible

subroutine lambda_of(Q, B, lam_quat, lam_det, co)
    real(dp), intent(in) :: Q(:, :), B(:, :)
    real(dp), intent(out) :: lam_quat, lam_det, co(3,3)
    real(dp) :: q1(4), q2(4), q3(4), p(4)
    integer :: i, j
    character(len=96) :: reason
    logical :: adm
    call lambda_of_admissible(Q, B, adm, reason)
    if (.not. adm) error stop 'lambda_of: '//trim(reason)
    do i = 1, 3
        do j = 1, 3
            co(i,j) = dot_product(Q(i,:), B(j,:)) ! admitted unit rows; Cauchy bound keeps this O(1), no overflow
        end do
    end do
    q1 = [0.0_dp, co(1,:)]; q2 = [0.0_dp, co(2,:)]; q3 = [0.0_dp, co(3,:)]
    p = qmul(qmul(q1, q2), q3)
    lam_quat = p(1); lam_det = -det3(co)
end subroutine lambda_of

! The kernel does not grade. It reports whether the axes are independent, and
! therefore whether a unique intersection exists at all.
subroutine kernel_determine(M, verdict, lam, detR, kapR, idim, why, margin)
    real(dp), intent(in) :: M(:, :)
    integer, intent(out) :: verdict, idim
    real(dp), intent(out), optional :: margin
    real(dp), intent(out) :: lam, detR, kapR
    character(len=*) , intent(out) :: why
    real(dp) :: sep
    real(dp) :: Q(3,size(M,2)), B(3,size(M,2)), R(3,3), co(3,3)
    real(dp) :: lq, ld, um, epsf, kgate
    integer :: n
    logical :: ok, bok
    ! EVERY output is defined here, before any test can return. The shape and
    ! size tests follow, and no arithmetic touches n until n has been screened,
    ! because the conditioning gate divides by it.
    lam = 0.0_dp; detR = 0.0_dp; kapR = 0.0_dp; idim = -1; sep = 0.0_dp
    verdict = OPENV; why = 'uninitialised'
    if (present(margin)) margin = 0.0_dp

```

```

n = size(M,2); um = epsilon(1.0_dp)
if (size(M,1) /= 3) then
    why = 'axis count is not three; the residence is not the one this kernel reads'
    return
end if
if (n < 4) then
    why = 'dimensional shortfall'; return
end if
if (.not. finite_mat(M)) then
    why = 'a row carries a non-quantity; nothing to read'; return
end if
epsf = 100.0_dp*um*real(n,dp)
kgate = min(1.0e6_dp, sqrt(27.0_dp/(100.0_dp*um*real(n,dp))))
call prepare_rows(M, Q, ok)
if (.not. ok) then
    verdict = OPENV; why = 'an axis carries no content; nothing to intersect'; return
end if
R = matmul(Q, transpose(Q))
detR = det3(R)
! A correlation Gram is positive semi-definite by construction, so a
! negative determinant is a rounding artifact and never a quantity. It is
! set to exactly zero rather than reported as read. Zero here is the
! algebraic floor of the quantity and is not the collapse floor epsf,
! which is a separate threshold tested further down.
if (detR < 0.0_dp) detR = 0.0_dp
call intersection_dim(R, idim, sep)
if (present(margin)) margin = sep
! The collapse test runs BEFORE any basis is constructed. Gram-Schmidt on a
! dependent triad divides by a residual nothing bounds below, so a basis
! built here would be rounding noise carried forward into the scalar. The
! order is the guard: a residence that determines no point never reaches
! the orthogonalisation at all.
! Existence is decided by the DIMENSION and never by the determinant,
! because the dimension is the spine's own quantity and the determinant is
! a magnitude read off it. Where the two floors disagree the verdict is
! neither broken nor determined: it is arithmetic at the edge, and it
! escalates rather than guessing which floor to believe.
if (idim > 0) then
    verdict = BROKEN
    select case (idim)
    case (1);      why = 'rank two; the intersection is a line, not a point'
    case (2);      why = 'rank one; the intersection is a plane, not a point'
    case default;  why = 'rank zero; no axis carries an independent direction'
    end select
    return
end if
if (detR <= epsf) then
    verdict = OPENV
    why = 'engineering-incomplete: rank floor and collapse floor disagree; re-run at higher precision'
    return
end if
call span_basis(Q, B, bok)
if (.not. bok) then
    ! Reaching here means the rank floor and the collapse floor both said
    ! three independent axes while Gram-Schmidt found a null residual. That
    ! is a third floor disagreeing with two, not a finding of dependence,
    ! and it escalates for the same reason the determinant disagreement
    ! does. Deciding BROKEN here would let the weakest instrument overrule
    ! the two that already agreed.
    verdict = OPENV
    why = 'engineering-incomplete: orthogonalisation residual contradicts the rank floor'
    return
end if
call lambda_of(Q, B, lq, ld, co)
! The two forms are computed by disjoint routes, the Hamilton product and
! the signed frame determinant, so their agreement is a live cross-check
! and not decoration. Disagreement is arithmetic failure and escalates.
if (abs(lq - ld) > 100.0_dp*epsilon(1.0_dp)*max(abs(ld), 1.0_dp)) then
    verdict = OPENV
    why = 'engineering-incomplete: quaternion and determinant forms of lambda disagree'
    return
end if
lam = lq
kapR = cond_sym3(R)
if (kapR >= kgate) then
    ! The intersection exists and is unique by the geometry. The instrument
    ! declines to report it, because at this conditioning the computed point
    ! is not separable from rounding. Instrument-silence about reach, never
    ! a claim that the intersection is absent or partial.
    verdict = OPENV
    why = 'engineering-incomplete: determination present, instrument declines'
    return
end if
verdict = DETERMINED
why = 'three independent axes; the intersection exists and is unique'
if (present(margin)) margin = sep
end subroutine kernel_determine

!=====

```

```

! SEAL L, THE TONGUE. Loads FIRST. Carries the direction the Number cannot,
! and closes the terms the row cascade later reads.
!=====

subroutine seal_L(slots, verdict, why)
  integer, intent(in) :: slots(:, :)
  integer, intent(out) :: verdict
  character(len=*) , intent(out) :: why
  integer :: i, j, a, b, nfull, ns
  ns = size(slots,1)
  ! the shelf is read, never remembered
  ! Zero is the empty cell and literals are positive. A negative id is not
  ! vocabulary and is refused before any slot is counted.
  if (any(slots < 0)) then
    verdict = BROKEN; why = 'negative literal id; not vocabulary'; return
  end if
  nfull = 0
  do i = 1, ns
    if (any(slots(i,:) /= 0)) nfull = nfull + 1
  end do
  if (nfull /= 3) then
    verdict = BROKEN
    write(why, '(A,I0,A)') 'deletion test returns ', nfull, ' slots, not three'
    return
  end if
  ! A slot is a vocabulary set. A repeated literal inside one slot is a
  ! malformed set, and admitting it would let hostile input reach a sealed
  ! verdict through a slot that was never well formed.
  do i = 1, ns
    do a = 1, size(slots,2)
      do b = a+1, size(slots,2)
        if (slots(i,a) /= 0 .and. slots(i,a) == slots(i,b)) then
          verdict = BROKEN
          write(why, '(A,I0,A)') 'repeated literal within slot ', i
          return
        end if
      end do
    end do
  end do
  do i = 1, ns
    do j = i+1, ns
      do a = 1, size(slots,2)
        do b = 1, size(slots,2)
          if (slots(i,a) /= 0 .and. slots(i,a) == slots(j,b)) then
            verdict = BROKEN
            write(why, '(A,I0,A,I0)') 'LIT collision, slot ', i, ' and slot ', j
            return
          end if
        end do
      end do
    end do
  end do
  verdict = SEALED; why = 'three slots, vocabulary pairwise disjoint'
end subroutine seal_L

! The six drills, routed to and not improvised. D3 carries the provenance
! flag and is mandatory for the closure-rowed compartment: a standing
! definition shaped at the Number makes rows-and-closure coincidence
! circular, and no other gate in the architecture reaches it.
subroutine drill_screen(d1, d2, d3_filed, d3_shaped, d4, d5, d6, ok, grade_cap, why)
  logical, intent(in) :: d1, d2, d3_filed, d3_shaped, d4, d5, d6
  logical, intent(out) :: ok
  character(len=*) , intent(out) :: grade_cap, why
  ok = .false.; grade_cap = 'none'
  if (.not. (d1 .and. d2)) then
    why = 'D1/D2 fail: Seal L break, terminal whatever the determinant reads'; return
  end if
  if (.not. d3_filed) then
    why = 'D3 fail: provenance unfiled; Number-shaped content in the Tongue seat'; return
  end if
  if (.not. d4) then
    why = 'D4 fail: a magnitude carried on the Tongue; sign only, veto never substitution'; return
  end if
  if (.not. d5) then
    why = 'D5 fail: register undeclared; re-index rather than refute'; return
  end if
  if (.not. d6) then
    why = 'D6 fail: the Tongue reproduced the blindness it was appointed to repair'; return
  end if
  ok = .true.
  ! The flag records; it does not bar. What it costs is a tier.
  if (d3_shaped) then
    grade_cap = 'structural'
    why = 'drills pass; provenance filed and SHAPED, so the assignment caps at structural'
  else
    grade_cap = 'theorem'
    why = 'drills pass; provenance filed and unshaped, no tier discount'
  end if
end subroutine drill_screen

```

```

!=====
! THE ROW CASCADE. One floor, three routers folded into their arguments,
! four admission gates. The count is forced by the questions: one about
! population, one about row identity, two about closure leaks.
!=====

subroutine row_cascade(has_content, faces_scoped, canonical, forward_mode, &
                     populated, worldly_row, drills_ok, frame_closed, &
                     compartment, why)
  logical, intent(in) :: has_content, faces_scoped, canonical, forward_mode
  logical, intent(in) :: populated, worldly_row, drills_ok, frame_closed
  integer, intent(out) :: compartment
  character(len=*), intent(out) :: why
  compartment = NOT_COMPARTMENTED
  if (.not. has_content) then
    why = 'F-0: every term decoration; contentless, refused, not compartmented'; return
  end if
  if (.not. faces_scoped) then
    why = 'R-1: faces not enumerated; a composite carries one compartment per face'; return
  end if
  if (.not. canonical) then
    why = 'R-2: not canonicalised; the compartment attaches to the stripped string'; return
  end if
  ! Population is asked before mode, because an unpopulated forward
  ! proposition has no lock to route and typing it world-rowed would assign a
  ! compartment to an object that determined nothing.
  if (.not. populated) then
    compartment = UNPOPULATED
    why = 'S-1: an axis is empty; two planes meet in a line; no lock forms'
    return
  end if
  if (forward_mode) then
    compartment = WORLD_ROWED
    why = 'R-3: a dated axis must be sourced, never measured; world-rowed unconditionally'
    return
  end if
  if (worldly_row) then
    compartment = WORLD_ROWED
    why = 'S-2: a row furnished by the world; the lock is total given those rows'
    return
  end if
  if (.not. drills_ok) then
    compartment = WORLD_ROWED
    why = 'S-3: term closure unscreened; typed by its remaining rows'
    return
  end if
  if (.not. frame_closed) then
    compartment = WORLD_ROWED
    why = 'S-4: existential import or transport; the leak names the compartment'
    return
  end if
  compartment = CLOSURE_ROWED
  why = 'rows are the closure; the two determinations are one; RA-conditional at the act'
end subroutine row_cascade

pure function compartment_grade(c, cap) result(s)
  integer, intent(in) :: c
  character(len=*), intent(in) :: cap
  character(len=64) :: s
  select case (c)
    case (CLOSURE_ROWED); s = 'row-indefeasible, defeasible against fault, cap '//trim(cap)
    case (UNPOPULATED); s = 'no determination to defeat; an axis is owed'
    case (WORLD_ROWED); s = 'revisable entirely in the rows; direction bought with the arrow'
    case default; s = 'no compartment'
  end select
end function compartment_grade

!=====
! THE GROUND AND THE ROUTER BIT.
!=====

! The router bit is measured at machine zero and never by exact equality. An
! eigenvalue arriving from a solver is 1 - 0(eps), and an equality test would
! silently return the wrong Ground dimension and route the candidate to the
! wrong halt protocol. The floor is stated, not assumed.
pure function ground_dim(diagv) result(d)
  real(dp), intent(in) :: diagv(:)
  integer :: d, i
  real(dp) :: floor_
  d = -1
  ! the router bit is unmeasured on a non-quantity
  if (.not. finite_vec(diagv)) return
  floor_ = real(max(size(diagv),1),dp) * epsilon(1.0_dp)
  d = 0
  do i = 1, size(diagv)
    if (abs(diagv(i) - 1.0_dp) <= floor_) d = d + 1
  end do
end function ground_dim

pure function route_halt(gdim) result(branch)

```

```

integer, intent(in) :: gdim
character(len=52) :: branch
if (gdim == 1) then
    branch = 'B.14.Xi    eight gates, blindness in the reader'
else if (gdim == 0) then
    branch = 'B.14.0    five gates, walls in the terrain'
else
    branch = 'terrain unmeasured; nothing emitted'
end if
end function route_halt

end module trisduction_rows

!=====
! EXECUTED. Nothing below is narrated; every figure is computed in this run.
!=====
program thesis_rows
    use trisduction_rows
    implicit none
    integer, parameter :: N = 24
    real(dp), parameter :: PI = 3.141592653589793238462643_dp
    real(dp) :: t(N), M(3,N), Mr(3,N), Two(3,N), Ret(3,N)
    real(dp) :: Q(3,N), B(3,N), co(3,3), R(3,3), Rr(3,3)
    real(dp) :: lam, detR, kapR, lam2, d2, lq, ld, marg
    real(dp) :: i4(4), j4(4), k4(4), p4(4), sig(4), dia(4)
    integer :: v, vL, idim, comp, i
    integer :: fails, checks
    logical :: ok, dok
    character(len=110) :: why, cap
    integer :: slots_ok(3,3)

    write(*, '(A)') repeat(' ', 78)
    write(*, '(A)') ' TRISDUCTION . RAM -- THE CORE THESIS, EXECUTED'
    write(*, '(A)') repeat(' ', 78)

    do i = 1, N
        t(i) = 2.0_dp*PI*real(i-1,dp)/real(N,dp)
    end do

    !----- THE SPINE, COMPUTED
    ! Three independent axes: the intersection has dimension zero. One point.
    ! Two axes: dimension one. A line. No point is determined at all.
    M(1,:) = sin(t)
    M(2,:) = cos(2.0_dp*t) + 0.35_dp*sin(t)
    M(3,:) = sin(3.0_dp*t) + 0.20_dp*cos(2.0_dp*t) + 0.12_dp*sin(t)
    Two = M; Two(3,:) = 0.6_dp*M(1,:) + 0.4_dp*M(2,:) ! third axis not independent

    write(*, '(/,A)') ' THE SPINE: WHAT A LOCK DETERMINES'
    call kernel_determine(M, v, lam, detR, kapR, idim, why, marg)
    write(*, '(A,A)') ' three independent axes : ', trim(token(v))
    write(*, '(A,A)') ' why = ', trim(why)
    write(*, '(A,I0,A)') ' intersection dimension = ', idim, ' -> exactly one point, no interval'
    write(*, '(A,F16.12)') ' det(R) = ', detR
    write(*, '(A,ES12.3,A,ES10.3,A,L1)') ' |lambda-2 - det(R)| = ', abs(lam*lam - detR), &
        ' bound ', 4.0_dp*kapR*epsilon(1.0_dp), ' within: ', &
        abs(lam*lam - detR) <= 4.0_dp*kapR*epsilon(1.0_dp)
    write(*, '(A,ES12.3,A)') ' rank margin = ', marg, ' smallest kept eigenvalue over tol'
    call kernel_determine(Two, v, lam2, d2, kapR, idim, why, marg)
    write(*, '(A,A)') ' third axis dependent : ', trim(token(v))
    write(*, '(A,A)') ' why = ', trim(why)
    write(*, '(A,I0,A)') ' intersection dimension = ', idim, ' -> a line; two planes meet in a line'
    write(*, '(A,ES12.3,A)') ' rank margin = ', marg, ' smallest RETAINED eigenvalue over tol; the third fell below'
    write(*, '(A)') ' A lock is not graded. It either determines a point or it does not exist.'

    !----- ORIENTATION, NOT STRENGTH
    ! The Number cannot read the sign. That is a fact about DIRECTION and never
    ! about whether the determination is complete.
    ! The exhibit refuses to print rather than printing from an unusable frame.
    call prepare_rows(M, Q, ok)
    if (.not. ok) error stop 'orientation exhibit: rows carry no variance'
    call span_basis(Q, B, ok)
    if (.not. ok) error stop 'orientation exhibit: no independent frame'
    call lambda_of(Q, B, lq, ld, co); R = matmul(Q, transpose(Q))
    Mr = -M
    ! B is deliberately NOT rebuilt: the reflected rows are read against the
    ! frame fixed once, which is what makes the comparison a comparison.
    call prepare_rows(Mr, Q, ok)
    if (.not. ok) error stop 'orientation exhibit: reflected rows carry no variance'
    call lambda_of(Q, B, lam2, d2, co)
    Rr = matmul(Q, transpose(Q))
    write(*, '(/,A)') ' WHAT THE NUMBER CANNOT READ: DIRECTION, NOT COMPLETENESS'
    write(*, '(A,F16.12,A,F16.12)') ' lambda(P) = ', lq, ' lambda(not P) = ', lam2
    write(*, '(A,ES12.3)') ' max|G(P) - G(not P)| = ', maxval(abs(R - Rr))
    write(*, '(A,ES12.3)') ' |det(R)(P) - det(R)(not P)| = ', abs(det3(R) - det3(Rr))
    write(*, '(A)') ' The Intersection is equally unique either way. The Tongue carries the sign.'

    i4 = [0.0_dp, 1.0_dp, 0.0_dp, 0.0_dp]; j4 = [0.0_dp, 0.0_dp, 1.0_dp, 0.0_dp]
    k4 = [0.0_dp, 0.0_dp, 0.0_dp, 1.0_dp]; p4 = qmul(qmul(i4, j4), k4)

```

```

write(*,'(A,F14.10)') '    substrate chirality Re(i*j*k)    = ', p4(1)

Ret(1,:) = sin(t); Ret(2,:) = cos(t); Ret(3,:) = sin(2.0_dp*t)
call kernel_determine(Ret, v, lam, detR, kapR, idim, why)
write(*,'(A,A)') '    the Return    : ', trim(token(v))
write(*,'(A,A)') '    why          = ', trim(why)
if (v == DETERMINED) then
    write(*,'(A,F16.12,A,F16.12)') '    det(R) = ', detR, '    |lambda| = ', abs(lam)
end if

!----- THE GROUND
sig = [1.0_dp,-1.0_dp,-1.0_dp,-1.0_dp]; dia = [-1.0_dp,-1.0_dp,-1.0_dp,-1.0_dp]
write(*,'(/,A)') ' THE GROUND AND THE ROUTER BIT'
write(*,'(A,I0,A,A)') '    sigma Ground dim = ', ground_dim(sig), ' -> ', trim(route_halt(ground_dim(sig)))
write(*,'(A,I0,A,A)') '    -I    Ground dim = ', ground_dim(dia), ' -> ', trim(route_halt(ground_dim(dia)))

!----- SEAL L AND THE DRILLS
slots_ok = reshape([101,102,0, 201,202,0, 301,302,0],[3,3],order=[2,1])
call seal_L(slots_ok, vL, why)
write(*,'(/,A)') ' SEAL L AND THE SIX DRILLS'
write(*,'(A,A,A,A)') '    Seal L    : ', trim(token(vL)), ' :: ', trim(why)
call drill_screen(.true.,.true.,.true.,.false.,.true.,.true.,.true.,.true., dok, cap, why)
write(*,'(A,L1,A,A,A,A)') '    unshaped terms: ', dok, '    cap=', trim(cap), ' :: ', trim(why)
call drill_screen(.true.,.true.,.true.,.true.,.true.,.true.,.true., dok, cap, why)
write(*,'(A,L1,A,A,A,A)') '    shaped terms : ', dok, '    cap=', trim(cap), ' :: ', trim(why)
call drill_screen(.true.,.true.,.false.,.true.,.true.,.true.,.true., dok, cap, why)
write(*,'(A,L1,A,A,A,A)') '    unfiled flag : ', dok, '    cap=', trim(cap), ' :: ', trim(why)

!----- THE ROW CASCADE
write(*,'(/,A)') ' THE ROW CASCADE'
call row_cascade(.true.,.true.,.true.,.false.,.true.,.false.,.true.,.true., comp, why)
write(*,'(A,A)') '    a closed residence has no outside -> ', trim(compartment_name(comp))
write(*,'(A,A)') '    ', trim(why)
write(*,'(A,A)') '    ', trim(compartment_grade(comp,'theorem'))

call row_cascade(.true.,.true.,.true.,.false.,.false.,.false.,.true.,.true., comp, why)
write(*,'(A,A)') '    every zero lies on the critical line -> ', trim(compartment_name(comp))
write(*,'(A,A)') '    ', trim(why)

call row_cascade(.true.,.true.,.true.,.false.,.true.,.true.,.true.,.true., comp, why)
write(*,'(A,A)') '    confined existents carry positive E_k -> ', trim(compartment_name(comp))
write(*,'(A,A)') '    ', trim(why)
write(*,'(A,A)') '    ', trim(compartment_grade(comp,'theorem'))

call row_cascade(.true.,.true.,.true.,.true.,.true.,.false.,.true.,.true., comp, why)
write(*,'(A,A)') '    any dated forward projection -> ', trim(compartment_name(comp))
write(*,'(A,A)') '    ', trim(why)

call row_cascade(.true.,.true.,.true.,.false.,.true.,.false.,.true.,.false., comp, why)
write(*,'(A,A)') '    this man is a bachelor -> ', trim(compartment_name(comp))
write(*,'(A,A)') '    ', trim(why)

call row_cascade(.false.,.true.,.true.,.false.,.true.,.false.,.true.,.true., comp, why)
write(*,'(A,A)') '    a contentless tautology -> ', trim(compartment_name(comp))
write(*,'(A,A)') '    ', trim(why)

write(*,'(/,A)') repeat('=',78)
write(*,'(A)') ' A lock is always a determination and is total about its object.'
write(*,'(A)') ' Its object is its intersection. The compartment names what the rows were.'
write(*,'(A)') ' Compartment III pays revisability for direction. Compartment I has neither.'
write(*,'(A)') ' Neither is the better lock. They are locks on different rows. Delta-M = 0.'
write(*,'(A)') repeat('=',78)

!----- THE BATTERY, PROOF OF LOAD
call battery()

contains

! Eight fuzz classes: seven over the numerical kernel, one over every
! symbolic and router public interface, then the oracle. Every check is a
! computed logical. The program stops here on any failure, so the final
! line of output is itself the receipt.
subroutine battery()
    use, intrinsic :: ieee_arithmetic
    integer, parameter :: NB = 8
    real(dp) :: H, T, nanv, pinf, ninf
    real(dp) :: mags(11)
    real(dp) :: M(3,NB), Q(3,NB), B(3,NB), G(3,3), A(3,3), w(3), sep
    real(dp) :: lam, detR, kapR, marg, lq, ld, co(3,3), q4(4), r4(4)
    real(dp) :: dl, ds(5), etas(6), scales(5), rho(3), tt(NB)
    integer :: i, j, k, v, idim
    character(len=160) :: why
    logical :: ok

    fails = 0; checks = 0
    H = sqrt(huge(1.0_dp)); T = sqrt(tiny(1.0_dp))
    nanv = ieee_value(1.0_dp, ieee_quiet_nan)
    pinf = ieee_value(1.0_dp, ieee_positive_inf)

```

```

ninf = ieee_value(1.0_dp, ieee_negative_inf)
mags = [0.0_dp, tiny(1.0_dp), T, epsilon(1.0_dp), 1.0_dp, 1.0_dp/epsilon(1.0_dp), H, &
        huge(1.0_dp)/4.0_dp, huge(1.0_dp), -1.0_dp, -H]
do i = 1, NB
    tt(i) = 6.283185307179586_dp*real(i-1,dp)/real(NB,dp)
end do

!----- CLASS 1
! Non-finite hostile inputs into every pure entry: NaN or .false. out.
call assert(.not. ieee_is_finite(snrm([1.0_dp, nanv])), 'snrm NaN -> NaN')
call assert(.not. ieee_is_finite(snrm([1.0_dp, pinf])), 'snrm +Inf -> NaN')
G = 1.0_dp; G(2,2) = ninf
call assert(.not. ieee_is_finite(det3(G)), 'det3 -Inf -> NaN')
call assert(.not. det3_representable(G), 'det3_representable -Inf -> F')
G(2,2) = nanv
call assert(.not. det3_representable(G), 'det3_representable NaN -> F')
q4 = [nanv,0.0_dp,0.0_dp,0.0_dp]; r4 = [1.0_dp,0.0_dp,0.0_dp,0.0_dp]
call assert(.not. all(ieee_is_finite(qmul(q4,r4))), 'qmul NaN -> NaN')
call assert(ground_dim([1.0_dp,pinf,-1.0_dp,-1.0_dp]) == -1, 'ground_dim Inf -> unmeasured')
! kernel: a non-finite row must route OPEN with a mechanism, never SEALED/BROKEN
M(1,:) = sin(tt); M(2,:) = cos(2.0_dp*tt); M(3,:) = sin(3.0_dp*tt); M(2,4) = nanv
call kernel_determine(M, v, lam, detR, kapR, idim, why, marg)
call assert(v == OPENV .and. index(why,'non-quantity') > 0, 'kernel NaN row -> OPEN, mechanism named')
M(2,4) = pinf
call kernel_determine(M, v, lam, detR, kapR, idim, why, marg)
call assert(v == OPENV, 'kernel Inf row -> OPEN')

!----- CLASS 2
! Extreme finite magnitudes: no overflow, no false PSD failure, no NaN after unscale.
do i = 1, 11
    G = 0.0_dp; G(1,1) = abs(mags(i)); G(2,2) = abs(mags(i)); G(3,3) = abs(mags(i))
    if (abs(mags(i)) > 0.0_dp .and. abs(mags(i)) < huge(1.0_dp)) then
        call intersection_dim(G, idim, sep)
        call assert(idim == 0, 'scaled identity rank 3 at magnitude index '//itoa(i))
    end if
end do
do i = 1, NB
    Q(1,i) = H*sin(tt(i)); Q(2,i) = H*cos(2.0_dp*tt(i)); Q(3,i) = H*sin(3.0_dp*tt(i))
end do
call span_basis(Q, B, ok)
call assert(ok .and. finite_mat(B), 'span_basis at sqrt(huge): finite frame')
do i = 1, NB
    Q(1,i) = T*sin(tt(i)); Q(2,i) = T*cos(2.0_dp*tt(i)); Q(3,i) = T*sin(3.0_dp*tt(i))
end do
call span_basis(Q, B, ok)
call assert(ok .and. finite_mat(B), 'span_basis at sqrt(tiny): finite frame')
A = 0.0_dp; A(1,1) = H; A(2,2) = T; A(3,3) = 1.0_dp
call jacobi3(A, w)
call assert(finite_vec(w), 'jacobi3 diag(H,T,1) finite')
A = H; call jacobi3(A, w)
call assert(finite_vec(w), 'jacobi3 all-H finite (3H fits)')
A = huge(1.0_dp)/2.0_dp; call jacobi3(A, w)
call assert(.not. finite_vec(w), 'jacobi3 all-huge/2: unrepresentable reported as Inf')
! det3 equilibration
G = 0.0_dp; G(1,1)=1.0e200_dp; G(2,2)=1.0e200_dp; G(3,3)=1.0e-200_dp
call assert(abs(det3(G)/1.0e200_dp - 1.0_dp) < 1.0e-12_dp, 'det3 diag(1e200,1e200,1e-200) = 1e200')
call assert(det3_representable(G), 'and representable')
G = 0.0_dp; G(1,1)=1.0e308_dp; G(2,2)=1.0e308_dp; G(3,3)=1.0e-20_dp
call assert(.not. det3_representable(G), 'det3 diag(1e308,1e308,1e-20) NOT representable')
G = 0.0_dp; G(1,1)=H; G(2,2)=T; G(3,3)=1.0_dp
call assert(det3_representable(G) .and. abs(det3(G)-H*T) <= 1.0e-12_dp*H*T, &
    'det3 diag(H,T,1) = H*T')

!----- CLASS 3
! Near-degenerate rank transitions, and scale stability of the classification.
ds = [epsilon(1.0_dp), sqrt(epsilon(1.0_dp)), 1.0e-12_dp, 1.0e-9_dp, 1.0e-6_dp]
scales = [tiny(1.0_dp)*1.0e10_dp, T, 1.0_dp, H/16.0_dp, huge(1.0_dp)/16.0_dp]
do i = 1, 5
    dl = ds(i)
    etas = [0.0_dp, tiny(1.0_dp), epsilon(1.0_dp), dl*dl, dl, sqrt(dl)]
    do j = 1, 6
        G = reshape([1.0_dp, 0.0_dp, 0.0_dp, 1.0_dp, dl, 0.0_dp, 1.0_dp, 2.0_dp*dl, etas(j)], [3,3], order=[2,1])
        G = matmul(G, transpose(G))
        call intersection_dim(G, idim, sep)
        do k = 1, 5
            block
                real(dp) :: Gs(3,3); integer :: ids
                Gs = scales(k)*G
                if (finite_mat(Gs)) then
                    call intersection_dim(Gs, ids, sep)
                    call assert(ids == idim, 'rank scale-stable d='//itoa(i)//' e='//itoa(j)//' S='//itoa(k))
                end if
            end block
        end do
    end do
end do

!----- CLASS 4/5
! Signed zero and reflection: determinant sign changes only under odd reflection.

```

```

G = 0.0_dp; G(1,1)=1.0_dp; G(2,2)=1.0_dp; G(3,3)=1.0_dp
A = G; A(1,2) = -0.0_dp; A(2,1) = -0.0_dp; A(3,2) = -0.0_dp
call assert(abs(det3(A) - det3(G)) <= 0.0_dp, 'signed zero leaves det unchanged, exactly')
A = G; A(1,1) = -1.0_dp
call assert(abs(det3(A) + det3(G)) <= 0.0_dp, 'single reflection flips det, exactly')
A = G; A(1,1) = -1.0_dp; A(2,2) = -1.0_dp
call assert(abs(det3(A) - det3(G)) <= 0.0_dp, 'double reflection preserves det, exactly')
! quaternion chirality agrees with the determinant route on a random-ish frame
do i = 1, NB
  M(1,i) = sin(tt(i)); M(2,i) = cos(2.0_dp*tt(i)) + 0.3_dp*sin(tt(i)); M(3,i) = sin(3.0_dp*tt(i))
end do
call prepare_rows(M, Q, ok); call span_basis(Q, B, ok); call lambda_of(Q, B, lq, ld, co)
call assert(abs(lq - ld) <= 100.0_dp*epsilon(1.0_dp)*max(abs(ld),1.0_dp), &
  'lambda quaternion == determinant route')
Q(1,:) = -Q(1,:); call lambda_of(Q, B, lam, detR, co)
! Under the strict IEEE, no-reassociation build this battery is run with,
! this is exact and stays exact. IEEE negation, multiplication and
! addition are sign-symmetric bit for bit, and scale() is a power-of-two
! multiply, so negating one row negates the composed scalar with no
! rounding anywhere on the route. The boot's own layer two tests the same
! flip with ==, and the register records it as the made-zero: the sign is
! displaced, never annihilated. A tolerance here would concede rounding
! the arithmetic does not commit.
call assert(abs(lam + lq) <= 0.0_dp, 'reflecting one row inverts lambda, exactly')

!----- CLASS 5b
! Hostile rows against lambda_of, in-process through its own predicate.
! The cancelling row [H,H,-H,-H]/2 has scaled norm H and must be refused at
! the unit gate before any coordinate dot could form.
block
  real(dp) :: Qh(3,NB), Bh(3,NB)
  character(len=96) :: rsn
  logical :: adm
  Qh = 0.0_dp; Bh = 0.0_dp
  Qh(1,1:4) = [H, H, -H, -H]/2.0_dp; Qh(2,5) = 1.0_dp; Qh(3,6) = 1.0_dp
  Bh(1,1) = 1.0_dp; Bh(2,2) = 1.0_dp; Bh(3,3) = 1.0_dp
  call lambda_of_admissible(Qh, Bh, adm, rsn)
  call assert(.not. adm .and. index(rsn,'Q rows are not unit') > 0, &
    'lambda_of refuses the cancelling H-row at the unit gate')
  Qh(1,:) = Bh(1,:); Qh(2,:) = Bh(2,:); Qh(3,:) = Bh(3,:)
  Bh(1,1:4) = [H, H, -H, -H]/2.0_dp
  call lambda_of_admissible(Qh, Bh, adm, rsn)
  call assert(.not. adm .and. index(rsn,'B rows are not unit') > 0, &
    'lambda_of refuses the cancelling H-row in B at the unit gate')
  Bh(1,:) = 0.0_dp; Bh(1,1) = nanv
  call lambda_of_admissible(Qh, Bh, adm, rsn)
  call assert(.not. adm .and. index(rsn,'not finite') > 0, &
    'lambda_of refuses NaN before any arithmetic')
  Bh(1,:) = 0.0_dp; Bh(1,1) = 1.0_dp
  call lambda_of_admissible(Qh, Bh, adm, rsn)
  call assert(adm, 'lambda_of admits an orthonormal identity frame')
end block
! qmul at the band the norm bound would have refused and the components fit
q4 = [0.0_dp, huge(1.0_dp)/4.0_dp, 0.0_dp, 0.0_dp]
r4 = [0.0_dp, 0.0_dp, 2.0_dp, 0.0_dp]
block
  real(dp) :: c4(4)
  c4 = qmul(q4, r4)
  call assert(finite_vec(c4), 'qmul: (huge/4)i * 2j is representable and returned finite')
  call assert(abs(c4(4) - huge(1.0_dp)/2.0_dp) <= epsilon(1.0_dp)*huge(1.0_dp), &
    'qmul: (huge/4)i * 2j = (huge/2)k exactly to rounding')
  q4 = [0.0_dp, huge(1.0_dp)/2.0_dp, 0.0_dp, 0.0_dp]
  r4 = [0.0_dp, 0.0_dp, 4.0_dp, 0.0_dp]
  c4 = qmul(q4, r4)
  call assert(.not. finite_vec(c4), 'qmul: (huge/2)i * 4j = 2huge k is refused as NaN, never wrapped')
end block

!----- CLASS 8
! The symbolic and router procedures, fuzzed as the numeric kernel is: every
! token typed, every compartment named, every halt branch routed, every
! Seal L refusal reached, every drill failure routed, every cascade gate
! taken. A public interface with no computed check is not covered.
block
  integer :: sl(3,3), sl4(4,3), cmp, tk
  character(len=140) :: rsn, cap
  logical :: dok
  ! tokens: three states, two refinements, one kernel token, one dot, one untyped
  call assert(trim(token_class(SEALED)) == 'verdict state' .and. trim(token_class(BROKEN)) == 'verdict state' &
    .and. trim(token_class(OPENV)) == 'verdict state', 'token_class: the three states')
  call assert(trim(token_class(XIO)) == 'refinement inside openness' .and. &
    trim(token_class(OHO)) == 'refinement inside openness', 'token_class: both refinements inside openness')
  call assert(trim(token_class(DETERMINED)) == 'kernel geometric token', 'token_class: the kernel token')
  call assert(trim(token_class(DOTMARK)) == 'outside the economy', 'token_class: the dot outside the economy')
  call assert(trim(token_class(99)) == 'untyped' .and. index(token(99),'untyped') > 0, 'token_class/token: untyped')
  do tk = 1, 7
    call assert(len_trim(token(tk)) > 0, 'token: every token prints')
  end do
  ! compartments and the router
  call assert(index(compartment_name(CLOSURE_ROWED),'CLOSURE-ROWED') > 0 .and. &

```

```

        index(compartment_name(UNPOPULATED),'UNPOPULATED') > 0 .and. &
        index(compartment_name(WORLD_ROWED),'WORLD_ROWED') > 0 .and. &
        index(compartment_name(NOT_COMPARTMENTED),'not compartmented') > 0, 'compartment_name: all four')
call assert(index(route_halt(1),'Xi') > 0 .and. index(route_halt(0),'B.14.0') > 0 .and. &
        index(route_halt(-1),'unmeasured') > 0, 'route_halt: dim 1, dim 0, unmeasured')
call assert(index(route_halt(2),'unmeasured') > 0 .and. index(route_halt(7),'unmeasured') > 0, &
        'route_halt: unexpected positive dims are unmeasured')
call assert(index(compartment_grade(CLOSURE_ROWED,'theorem'),'theorem') > 0 .and. &
        index(compartment_grade(UNPOPULATED,'x'),'no determination') > 0 .and. &
        index(compartment_grade(WORLD_ROWED,'x'),'rows') > 0, 'compartment_grade: all three')
call assert(trim(compartment_grade(99,'x')) == 'no compartment', 'compartment_grade: default')
! Seal L: every refusal and the one admission
s1 = reshape([101,102,0, 201,202,0, 301,302,0],[3,3],order=[2,1])
call seal_L(s1, tk, rsn); call assert(tk == SEALED, 'seal_L: clean three slots seal')
s1 = reshape([101,102,0, 201,101,0, 301,302,0],[3,3],order=[2,1])
call seal_L(s1, tk, rsn); call assert(tk == BROKEN .and. index(rsn,'LIT') > 0, 'seal_L: cross-slot collision')
s1 = reshape([101,101,0, 201,202,0, 301,302,0],[3,3],order=[2,1])
call seal_L(s1, tk, rsn); call assert(tk == BROKEN .and. index(rsn,'repeated') > 0, 'seal_L: within-slot repeat')
s1 = reshape([101,-1,0, 201,202,0, 301,302,0],[3,3],order=[2,1])
call seal_L(s1, tk, rsn); call assert(tk == BROKEN .and. index(rsn,'negative') > 0, 'seal_L: negative id')
s1 = reshape([101,102,0, 0,0,0, 301,302,0],[3,3],order=[2,1])
call seal_L(s1, tk, rsn); call assert(tk == BROKEN .and. index(rsn,'2 slots') > 0, 'seal_L: two slots, not three')
s14 = reshape([101,0,0, 201,0,0, 301,0,0, 401,0,0],[4,3],order=[2,1])
call seal_L(s14, tk, rsn); call assert(tk == BROKEN .and. index(rsn,'4 slots') > 0, 'seal_L: four slots counted, not three')
! drills: each failure routes, both caps
call drill_screen(.false.,.true.,.true.,.false.,.true.,.true.,.true., dok, cap, rsn)
call assert(.not. dok .and. index(rsn,'D1/D2') > 0, 'drill: D1 fails')
call drill_screen(.true.,.false.,.true.,.false.,.true.,.true.,.true., dok, cap, rsn)
call assert(.not. dok .and. index(rsn,'D1/D2') > 0, 'drill: D2 fails')
call drill_screen(.true.,.true.,.true.,.false.,.true.,.true.,.true., dok, cap, rsn)
call assert(.not. dok .and. index(rsn,'D3') > 0, 'drill: unfiled provenance')
call drill_screen(.true.,.true.,.true.,.false.,.false.,.true.,.true., dok, cap, rsn)
call assert(.not. dok .and. index(rsn,'D4') > 0, 'drill: magnitude on the Tongue')
call drill_screen(.true.,.true.,.true.,.false.,.true.,.false.,.true., dok, cap, rsn)
call assert(.not. dok .and. index(rsn,'D5') > 0, 'drill: register undeclared')
call drill_screen(.true.,.true.,.true.,.false.,.true.,.true.,.false., dok, cap, rsn)
call assert(.not. dok .and. index(rsn,'D6') > 0, 'drill: blindness reproduced')
call drill_screen(.true.,.true.,.true.,.true.,.false.,.true.,.true., dok, cap, rsn)
call assert(dok .and. trim(cap) == 'theorem', 'drill: unshaped caps at theorem')
call drill_screen(.true.,.true.,.true.,.true.,.true.,.true.,.true., dok, cap, rsn)
call assert(dok .and. trim(cap) == 'structural', 'drill: shaped caps at structural')
! cascade: every gate taken in order
call row_cascade(.false.,.true.,.true.,.true.,.false.,.true.,.false.,.true.,.true., cmp, rsn)
call assert(cmp == NOT_COMPARTMENTED .and. index(rsn,'F-0') > 0, 'cascade: F-0 refuses')
call row_cascade(.true.,.false.,.true.,.false.,.true.,.false.,.true.,.true., cmp, rsn)
call assert(cmp == NOT_COMPARTMENTED .and. index(rsn,'R-1') > 0, 'cascade: R-1 faces')
call row_cascade(.true.,.true.,.false.,.false.,.true.,.false.,.true.,.true., cmp, rsn)
call assert(cmp == NOT_COMPARTMENTED .and. index(rsn,'R-2') > 0, 'cascade: R-2 canonical')
call row_cascade(.true.,.true.,.true.,.true.,.false.,.false.,.true.,.true., cmp, rsn)
call assert(cmp == UNPOPULATED .and. index(rsn,'S-1') > 0, 'cascade: unpopulated forward is II, not III')
call row_cascade(.true.,.true.,.true.,.true.,.true.,.true.,.false.,.true., cmp, rsn)
call assert(cmp == WORLD_ROWED .and. index(rsn,'R-3') > 0, 'cascade: populated forward is III')
call row_cascade(.true.,.true.,.true.,.false.,.true.,.true.,.true.,.true., cmp, rsn)
call assert(cmp == WORLD_ROWED .and. index(rsn,'S-2') > 0, 'cascade: worldly row is III')
call row_cascade(.true.,.true.,.true.,.false.,.true.,.false.,.false.,.true., cmp, rsn)
call assert(cmp == WORLD_ROWED .and. index(rsn,'S-3') > 0, 'cascade: unscreened terms route by rows')
call row_cascade(.true.,.true.,.true.,.false.,.true.,.false.,.true.,.false., cmp, rsn)
call assert(cmp == WORLD_ROWED .and. index(rsn,'S-4') > 0, 'cascade: frame leak names the compartment')
call row_cascade(.true.,.true.,.true.,.false.,.true.,.false.,.true.,.true., cmp, rsn)
call assert(cmp == CLOSURE_ROWED, 'cascade: closure-rowed reached')
! the router bit on constructed and solver-noisy diagonals
call assert(ground_dim([1.0_dp,-1.0_dp,-1.0_dp,-1.0_dp]) == 1 .and. &
        ground_dim([1.0_dp-2.0_dp*epsilon(1.0_dp),-1.0_dp,-1.0_dp,-1.0_dp]) == 1 .and. &
        ground_dim([-1.0_dp,-1.0_dp,-1.0_dp,-1.0_dp]) == 0, 'ground_dim: exact, noisy, and diagonal')
end block

!----- CLASS 6
! PSD boundary for cond_sym3: tolerance relative to spectral scale only.
A = 0.0_dp; A(1,1)=1.0_dp; A(2,2)=1.0_dp; A(3,3)=epsilon(1.0_dp)
call assert(ieee_is_finite(cond_sym3(A)) .or. cond_sym3(A) >= huge(1.0_dp), 'diag(1,1,-eps) tolerated')
A = 0.0_dp; A(1,1)=H; A(2,2)=H; A(3,3)=epsilon(1.0_dp)*H
call assert(cond_sym3(A) >= 0.0_dp, 'diag(H,H,-eps*H) tolerated relative to scale')
A = 0.0_dp; A(1,1)=1.0_dp; A(2,2)=1.0_dp; A(3,3)=0.0_dp
call assert(cond_sym3(A) >= huge(1.0_dp), 'diag(1,1,0) -> huge conditioning')

!----- CLASS 7
! Eigenvalue unscale across scales and structures.
rho = [1.0_dp-epsilon(1.0_dp), 1.0_dp-sqrt(epsilon(1.0_dp)), -1.0_dp+epsilon(1.0_dp)]
do k = 1, 5
  do i = 1, 3
    A = 0.0_dp; A(1,1)=1.0_dp; A(2,2)=1.0_dp; A(3,3)=epsilon(1.0_dp); A(1,2)=rho(i); A(2,1)=rho(i)
    A = scales(k)*A
    if (finite_mat(A)) then
      call jacobi3(A, w)
      call assert(finite_vec(w) .or. scales(k) > H, &
          'unscale finite where representable S='//itoa(k)//' rho='//itoa(i))
    end if
  end do
end do
end do

```

```

write(*,'(A)') repeat('= ',78)
write(*,'(A,I0,A,I0)') ' THE BATTERY   checks ', checks, '   failures ', fails
write(*,'(A)') repeat('= ',78)
if (fails > 0) error stop 'THE BATTERY FAILED: the load is not proven'
write(*,'(A)') ' The load is proven. A lock is always a determination. Delta-M = 0.'
end subroutine battery

subroutine assert(cond, label)
  logical, intent(in) :: cond
  character(len=*), intent(in) :: label
  checks = checks + 1
  if (.not. cond) then
    fails = fails + 1
    write(*,'(A,A)') ' FAIL ', label
  end if
end subroutine assert
function itoa(n) result(s)
  integer, intent(in) :: n
  character(len=12) :: s
  write(s,'(I0)') n
end function itoa

end program thesis_rows

```